

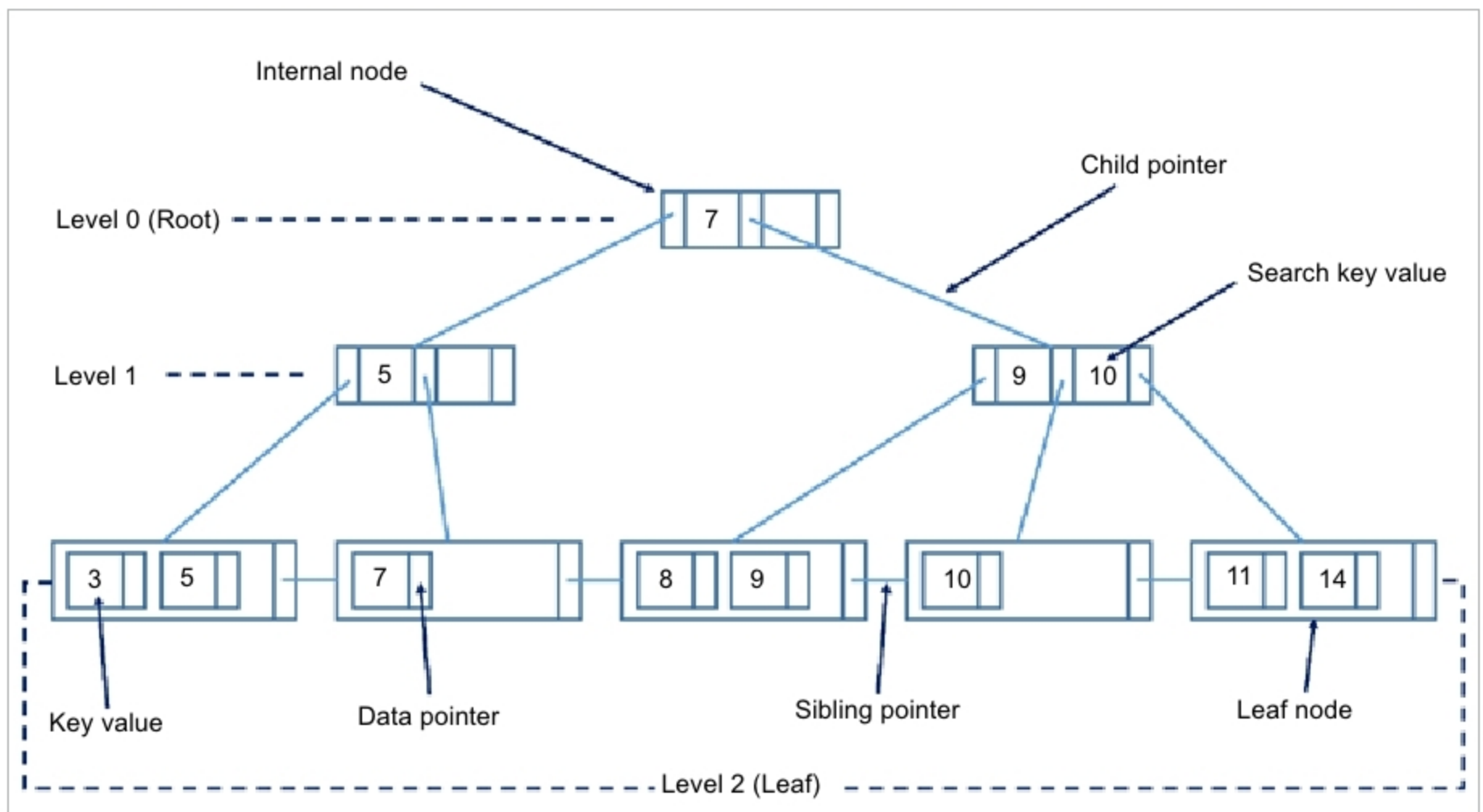


Figure 1: The B+ tree data structure

- It requires MariaDB server on Linux and Windows.
- MyRocks builds are available on platforms that support sufficiently modern compilers like Ubuntu Trusty, Xenial (amd64 and ppc64el), Ubuntu Yakkety (amd64), Debian Jessie (stable --amd64, ppc64el), Debian Stretch, Sid (testing and unstable -- amd64), CentOS/RHEL 7 (amd64), CentOS/RHEL 7.3 (amd64), Fedora 24 and 25 (amd64), OpenSUSE 42 (amd64), and Windows 64 (Zip and MSI).
- MariaDB's optimistic mode of parallel replication may not be supported.
- MyRocks is not available for 32-bit platforms.
- Galera Cluster is tightly integrated into the InnoDB storage engine (it also supports Percona's XtraDB which is a modified version of InnoDB). Galera Cluster does not work with any other storage engine like MyRocks (or TokuDB, for example).

Functionality and architecture

Being a database engine, MyRocks is used by the DBMS to create, read, update and delete (CRUD) data from a database. Information in a database is stored as bits laid out as data structures in a storage system that can be efficiently read from and written to the properties of hardware. There are various types of data structures — linear, trees, hash-based and graphs. Most database engines use trees for storing and retrieving data. The most commonly used data structure for trees is the B tree. Database engines use this type of data structure for storing/retrieving data. Some even use the Indexed Sequential Access Method (ISAM) but the B tree is more popular.

In a B tree storage system, keys and data are stored as nodes and it starts with a root node which is connected to other parent and child nodes. This process of data storage makes it easier for data insertion and deletion. However, the B+ tree is a newer extension of the B tree and it is more efficient for data insertion,

deletion and search operations. It consists of a root, internal nodes and leaves. The root may be either a leaf or a node with two or more children. In a B tree, both the data and keys are stored in the internal and leaf nodes, but the B+ tree can store data only in the leaf nodes, while the keys are stored in the internal nodes. The leaf nodes of a B+ tree are linked together like a single linked list and this makes the search operation more efficient.

The B+ tree is used to store large amounts of data that cannot be stored in the main memory. As the size of the main memory is often limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory, whereas leaf nodes (containing data) are stored in the secondary memory. The internal nodes of a B+ tree are also called index nodes because they always point to the leaf nodes that contain the actual data. A B+ tree (except the root node) can have $M/2$ minimum internal and leaf nodes (where 'M' is the maximum number of nodes except the root node) and can have $M - 1$ leaf nodes.